

```

Bulk_quote(const std::string& book, double price,
           std::size_t qty, double disc):
    Disc_quote(book, price, qty, disc) { }
// 覆盖基类中的函数版本以实现一种新的折扣策略
double net_price(std::size_t) const override;
};

```

这个版本的 `Bulk_quote` 的直接基类是 `Disc_quote`，间接基类是 `Quote`。每个 `Bulk_quote` 对象包含三个子对象：一个（空的）`Bulk_quote` 部分、一个 `Disc_quote` 子对象和一个 `Quote` 子对象。

如前所述，每个类各自控制其对象的初始化过程。因此，即使 `Bulk_quote` 没有自己的数据成员，它也仍然需要像原来一样提供一个接受四个参数的构造函数。该构造函数将它的实参传递给 `Disc_quote` 的构造函数，随后 `Disc_quote` 的构造函数继续调用 `Quote` 的构造函数。`Quote` 的构造函数首先初始化 `bulk` 的 `bookNo` 和 `price` 成员，当 `Quote` 的构造函数结束后，开始运行 `Disc_quote` 的构造函数并初始化 `quantity` 和 `discount` 成员，最后运行 `Bulk_quote` 的构造函数，该函数无须执行实际的初始化或其他工作。

611

关键概念：重构

在 `Quote` 的继承体系中增加 `Disc_quote` 类是重构（refactoring）的一个典型示例。重构负责重新设计类的体系以便将操作和/或数据从一个类移动到另一个类中。对于面向对象的应用程序来说，重构是一种很普遍的现象。

值得注意的是，即使我们改变了整个继承体系，那些使用了 `Bulk_quote` 或 `Quote` 的代码也无须进行任何改动。不过一旦类被重构（或以其他方式被改变），就意味着我们必须重新编译含有这些类的代码了。

15.4 节练习

练习 15.15：定义你自己的 `Disc_quote` 和 `Bulk_quote`。

练习 15.16：改写你在 15.2.2 节（第 533 页）练习中编写的数量受限的折扣策略，令其继承 `Disc_quote`。

练习 15.17：尝试定义一个 `Disc_quote` 的对象，看看编译器给出的错误信息是什么？



15.5 访问控制与继承

每个类分别控制自己的成员初始化过程（参见 15.2.2 节，第 531 页），与之类似，每个类还分别控制着其成员对于派生类来说是否可访问（accessible）。

受保护的成员

如前所述，一个类使用 `protected` 关键字来声明那些它希望与派生类分享但是不想被其他公共访问使用的成员。`protected` 说明符可以看做是 `public` 和 `private` 中和后的产物：

- 和私有成员类似，受保护的成员对于类的用户来说是不可访问的。

- 和公有成员类似，受保护的成员对于派生类的成员和友元来说是可访问的。

此外，protected 还有另外一条重要的性质。

- 派生类的成员或友元只能通过派生类对象来访问基类的受保护成员。派生类对于一个基类对象中的受保护成员没有任何访问特权。

为了解最后一条规则，请考虑如下的例子：

612

```
class Base {
protected:
    int prot_mem;           // protected 成员
};

class Sneaky : public Base {
    friend void clobber(Sneaky&);    // 能访问 Sneaky::prot_mem
    friend void clobber(Base&);      // 不能访问 Base::prot_mem
    int j;                          // j 默认是 private
};

// 正确: clobber 能访问 Sneaky 对象的 private 和 protected 成员
void clobber(Sneaky &s) { s.j = s.prot_mem = 0; }
// 错误: clobber 不能访问 Base 的 protected 成员
void clobber(Base &b) { b.prot_mem = 0; }
```

如果派生类（及其友元）能访问基类对象的受保护成员，则上面的第二个 clobber（接受一个 Base&）将是合法的。该函数不是 Base 的友元，但是它仍然能够改变一个 Base 对象的内容。如果按照这样的思路，则我们只要定义一个形如 Sneaky 的新类就能非常简单地规避掉 protected 提供的访问保护了。

要想阻止以上的用法，我们就要做出如下规定，即派生类的成员和友元只能访问派生类对象中的基类部分的受保护成员；对于普通的基类对象中的成员不具有特殊的访问权限。

公有、私有和受保护继承

某个类对其继承而来的成员的访问权限受到两个因素影响：一是在基类中该成员的访问说明符，二是在派生类的派生列表中的访问说明符。举个例子，考虑如下的继承关系：

```
class Base {
public:
    void pub_mem();           // public 成员
protected:
    int prot_mem;             // protected 成员
private:
    char priv_mem;           // private 成员
};

struct Pub_Derv : public Base {
    // 正确: 派生类能访问 protected 成员
    int f() { return prot_mem; }
    // 错误: private 成员对于派生类来说是不可访问的
    char g() { return priv_mem; }
};

struct Priv_Derv : private Base {
    // private 不影响派生类的访问权限
    int fl() const { return prot_mem; }
};
```

派生访问说明符对于派生类的成员（及友元）能否访问其直接基类的成员没什么影响。对基类成员的访问权限只与基类中的访问说明符有关。Pub_Derv 和 Priv_Derv 都能访问受保护的成员 `prot_mem`，同时它们都不能访问私有成员 `priv_mem`。

派生访问说明符的目的是控制派生类用户（包括派生类的派生类在内）对于基类成员的访问权限：

```
Pub_Derv d1;           // 继承自 Base 的成员是 public 的
Priv_Derv d2;          // 继承自 Base 的成员是 private 的
d1.pub_mem();          // 正确：pub_mem 在派生类中是 public 的
d2.pub_mem();          // 错误：pub_mem 在派生类中是 private 的
```

Pub_Derv 和 Priv_Derv 都继承了 `pub_mem` 函数。如果继承是公有的，则成员将遵循其原有的访问说明符，此时 `d1` 可以调用 `pub_mem`。在 Priv_Derv 中，Base 的成员是私有的，因此类的用户不能调用 `pub_mem`。

派生访问说明符还可以控制继承自派生类的新类的访问权限：

```
struct Derived_from_Public : public Pub_Derv {
    // 正确：Base::prot_mem 在 Pub_Derv 中仍然是 protected 的
    int use_base() { return prot_mem; }
};
struct Derived_from_Private : public Priv_Derv {
    // 错误：Base::prot_mem 在 Priv_Derv 中是 private 的
    int use_base() { return prot_mem; }
};
```

Pub_Derv 的派生类之所以能访问 Base 的 `prot_mem` 成员是因为该成员在 Pub_Derv 中仍然是受保护的。相反，Priv_Derv 的派生类无法执行类的访问，对于它们来说，Priv_Derv 继承自 Base 的所有成员都是私有的。

假设我们之前还定义了一个名为 Prot_Derv 的类，它采用受保护继承，则 Base 的所有公有成员在新定义的类中都是受保护的。Prot_Derv 的用户不能访问 `pub_mem`，但是 Prot_Derv 的成员和友元可以访问那些继承而来的成员。



派生类向基类转换的可访问性

派生类向基类的转换（参见 15.2.2 节，第 530 页）是否可访问由使用该转换的代码决定，同时派生类的派生访问说明符也会有影响。假定 D 继承自 B：

- 只有当 D 公有地继承 B 时，用户代码才能使用派生类向基类的转换；如果 D 继承 B 的方式是受保护的或者私有的，则用户代码不能使用该转换。
- 不论 D 以什么方式继承 B，D 的成员函数和友元都能使用派生类向基类的转换；派生类向其直接基类的类型转换对于派生类的成员和友元来说永远是可访问的。
- 如果 D 继承 B 的方式是公有的或者受保护的，则 D 的派生类的成员和友元可以使用 D 向 B 的类型转换；反之，如果 D 继承 B 的方式是私有的，则不能使用。



对于代码中的某个给定节点来说，如果基类的公有成员是可访问的，则派生类向基类的类型转换也是可访问的；反之则不行。

关键概念：类的设计与受保护的成员

不考虑继承的话，我们可以认为一个类有两种不同的用户：普通用户和类的实现者。

其中，普通用户编写的代码使用类的对象，这部分代码只能访问类的公有（接口）成员；实现者则负责编写类的成员和友元的代码，成员和友元既能访问类的公有部分，也能访问类的私有（实现）部分。

如果进一步考虑继承的话就会出现第三种用户，即派生类。基类把它希望派生类能够使用的部分声明成受保护的。普通用户不能访问受保护的成员，而派生类及其友元仍旧不能访问私有成员。

和其他类一样，基类应该将其接口成员声明为公有的；同时将属于其实现的部分分成两组：一组可供派生类访问，另一组只能由基类及基类的友元访问。对于前者应该声明为受保护的，这样派生类就能在实现自己的功能时使用基类的这些操作和数据；对于后者应该声明为私有的。

友元与继承

就像友元关系不能传递一样（参见 7.3.4 节，第 250 页），友元关系同样也不能继承。基类的友元在访问派生类成员时不具有特殊性，类似的，派生类的友元也不能随意访问基类的成员：

```
class Base {
    // 添加 friend 声明，其他成员与之前的版本一致
    friend class Pal;           // Pal 在访问 Base 的派生类时不具有特殊性
};
class Pal {
public:
    int f(Base b) { return b.prot_mem; } // 正确：Pal 是 Base 的友元
    int f2(Sneaky s) { return s.j; }     // 错误：Pal 不是 Sneaky 的友元
    // 对基类的访问权限由基类本身控制，即使对于派生类的基类部分也是如此
    int f3(Sneaky s) { return s.prot_mem; } // 正确：Pal 是 Base 的友元
};
```

如前所述，每个类负责控制自己的成员的访问权限，因此尽管看起来有点儿奇怪，但 f3 确实是正确的。Pal 是 Base 的友元，所以 Pal 能够访问 Base 对象的成员，这种可访问性包括了 Base 对象内嵌在其派生类对象中的情况。

615

当一个类将另一个类声明为友元时，这种友元关系只对做出声明的类有效。对于原来那个类来说，其友元的基类或者派生类不具有特殊的访问能力：

```
// D2 对 Base 的 protected 和 private 成员不具有特殊的访问能力
class D2 : public Pal {
public:
    int mem(Base b)
    { return b.prot_mem; }           // 错误：友元关系不能继承
};
```

Note

不能继承友元关系；每个类负责控制各自成员的访问权限。

改变个别成员的可访问性

有时我们需要改变派生类继承的某个名字的访问级别，通过使用 using 声明（参见 3.1 节，第 74 页）可以达到这一目的：

```
class Base {
public:
```

```

    std::size_t size() const { return n; }
protected:
    std::size_t n;
};
class Derived : private Base {           // 注意: private 继承
public:
    // 保持对象尺寸相关的成员的访问级别
    using Base::size;
protected:
    using Base::n;
};

```

因为 Derived 使用了私有继承, 所以继承而来的成员 size 和 n (在默认情况下) 是 Derived 的私有成员。然而, 我们使用 using 声明语句改变了这些成员的可访问性。改变之后, Derived 的用户将可以使用 size 成员, 而 Derived 的派生类将能使用 n。

通过在类的内部使用 using 声明语句, 我们可以将该类的直接或间接基类中的任何可访问成员 (例如, 非私有成员) 标记出来。using 声明语句中名字访问权限由该 using 声明语句之前的访问说明符来决定。也就是说, 如果一条 using 声明语句出现在类的 private 部分, 则该名字只能被类的成员和友元访问; 如果 using 声明语句位于 public 部分, 则类的所有用户都能访问它; 如果 using 声明语句位于 protected 部分, 则该名字对于成员、友元和派生类是可访问的。



派生类只能为那些它可以访问的名字提供 using 声明。

616

默认的继承保护级别

在 7.2 节 (第 240 页) 中我们曾经介绍过使用 struct 和 class 关键字定义的类型具有不同的默认访问说明符。类似的, 默认派生运算符也由定义派生类所用的关键字来决定。默认情况下, 使用 class 关键字定义的派生类是私有继承的; 而使用 struct 关键字定义的派生类是公有继承的:

```

class Base { /* ... */ };
struct D1 : Base { /* ... */ };           // 默认 public 继承
class D2 : Base { /* ... */ };           // 默认 private 继承

```

人们常常有一种错觉, 认为在使用 struct 关键字和 class 关键字定义的类型之间还有更深层次的差别。事实上, 唯一的差别就是默认成员访问说明符及默认派生访问说明符; 除此之外, 再无其他不同之处。



一个私有派生的类最好显式地将 private 声明出来, 而不要仅仅依赖于默认的设置。显式声明的好处是可以令私有继承关系清晰明了, 不至于产生误会。

15.5 节练习

练习 15.18: 假设给定了第 543 页和第 544 页的类, 同时已知每个对象的类型如注释所示, 判断下面的哪些赋值语句是合法的。解释那些不合法的语句为什么不被允许:

```

Base *p = &d1;           // d1 的类型是 Pub_Derv
p = &d2;                 // d2 的类型是 Priv_Derv

```

```

p = &d3;                // d3 的类型是 Prot_Derv
p = &ddl;                // ddl 的类型是 Derived_from_Public
p = &dd2;                // dd2 的类型是 Derived_from_Private
p = &dd3;                // dd3 的类型是 Derived_from_Protected

```

练习 15.19: 假设 543 页和 544 页的每个类都有如下形式的成员函数:

```
void memfcn(Base &b) { b = *this; }
```

对于每个类, 分别判断上面的函数是否合法。

练习 15.20: 编写代码检验你对前面两题的回答是否正确。

练习 15.21: 从下面这些一般性抽象概念中任选一个 (或者选一个你自己的), 将其对应的一组类型组织成一个继承体系:

- (a) 图形文件格式 (如 gif、tiff、jpeg、bmp)
- (b) 图形基元 (如方格、圆、球、圆锥)
- (c) C++ 语言中的类型 (如类、函数、成员函数)

练习 15.22: 对于你在上一题中选择的类, 为其添加合适的虚函数及公有成员和受保护的成员。

15.6 继承中的类作用域



每个类定义自己的作用域 (参见 7.4 节, 第 253 页), 在这个作用域内我们定义类的成员。当存在继承关系时, 派生类的作用域嵌套 (参见 2.2.4 节, 第 43 页) 在其基类的作用域之内。如果一个名字在派生类的作用域内无法正确解析, 则编译器将继续在外层的基类作用域中寻找该名字的定义。

派生类的作用域位于基类作用域之内这一事实可能有点儿出人意料, 毕竟在我们的程序文本中派生类和基类的定义是相互分离开来的。不过也恰恰因为类作用域有这种继承嵌套的关系, 所以派生类才能像使用自己的成员一样使用基类的成员。例如, 当我们编写下面的代码时:

```

Bulk_quote bulk;
cout << bulk.isbn();

```

名字 isbn 的解析将按照下述过程所示:

- 因为我们是通过 Bulk_quote 的对象调用 isbn 的, 所以首先在 Bulk_quote 中查找, 这一步没有找到名字 isbn。
- 因为 Bulk_quote 是 Disc_quote 的派生类, 所以接下来在 Disc_quote 中查找, 仍然找不到。
- 因为 Disc_quote 是 Quote 的派生类, 所以接着查找 Quote; 此时找到了名字 isbn, 所以我们使用的 isbn 最终被解析为 Quote 中的 isbn。

在编译时进行名字查找

一个对象、引用或指针的静态类型 (参见 15.2.3 节, 第 532 页) 决定了该对象的哪些成员是可见的。即使静态类型与动态类型可能不一致 (当使用基类的引用或指针时会发生